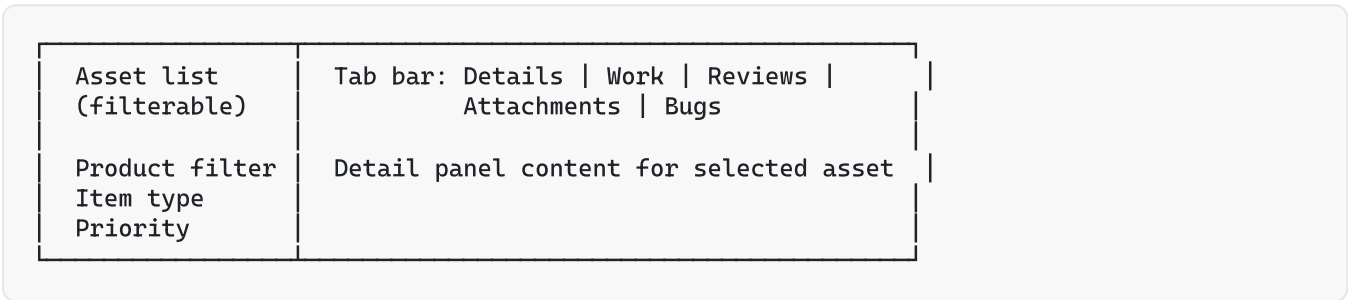


Asset Viewer

Last updated: 2026-05-11

The Asset Viewer is the central UI surface in ArtHound. It presents the studio's asset inventory in a three-panel layout: a filterable asset list on the left, and a tabbed detail panel on the right that combines source-synced field data, work tracking, reviews, and attachments into a single coherent view per asset.

Layout



The list panel shows configurable columns drawn from the view schema (see below). Selecting an asset populates the detail panel without a page navigation. Column visibility is persisted in `localStorage` per studio.

View Schema

The view schema drives both the asset list columns and the detail panel field groupings. It is fetched from `GET /api/assets/view-schema` and returned as:

```

{
  "columns": [
    {
      "id": "name",
      "label": "Name",
      "source": "slot",
      "slotKey": "name",
      "fieldType": "text",
      "defaultVisible": true,
      "metaBucket": "production",
      "displayTier": "primary"
    },
    {
      "id": "custom_field_abc",
      "label": "Complexity",
      "source": "meta",
      "fieldName": "Complexity",
      "fieldType": "singleSelect",
      "defaultVisible": false,
      "metaBucket": "technical",
      "displayTier": "secondary"
    },
    ...
  ]
}

```

Source values: - `"slot"` , a named ArtHound slot (name, dev_name, item_type, product, priority, project_date, status, asset_number, team) - `"meta"` , a raw source field stored in the `meta` JSONB column

Hidden-tier fields are excluded from the view schema entirely, they are never shown in the UI.

Display Tiers

Every field in the system has a `displayTier` :

Tier	Behaviour
<code>primary</code>	Shown by default in the Details tab
<code>secondary</code>	Hidden behind a "Show more" toggle in the Details tab
<code>hidden</code>	Never shown; excluded from view schema

Display tier is auto-derived from `metaBucket` when a mapping is first created, and can be manually overridden by studios in the field mapping UI. The mapping: Production → primary; Technical / Business / Custom → secondary; Source native → hidden.

Tabs

Details Tab (`DetailsTab.jsx`)

Displays the asset's field data in two sections:

Primary section (always visible): - All slot fields with values (name, item type, product, priority, etc.) - All `primary`-tier meta fields

Secondary section (collapsed by default): - All `secondary`-tier meta fields, revealed by a "Show N more field(s)" toggle

Field rendering: - Linked records are rendered as clickable references that open a drill-down modal (e.g., clicking a product name resolves the product record) - URL-type fields render as external links - Attachment-type fields are handled by the Attachments tab, not here - Empty fields show a dash (" ")

The Details tab reads from the `asset` object passed by the parent (loaded via `GET /api/assets/{id}`), cross-referenced against the view schema for tier assignments.

Work Tab (`WorkTab.jsx`)

Shows work items for the selected asset in two modes, toggleable by the user:

Source mode: work synced from the source tool (`replicated_work`). List view only (name, status, estimate). Read-only.

ArtHound mode: generated work snapshots (`generated_work`). Supports two views: - **List view:** name, craft, date range, estimate in days - **Timeline view:** Gantt-style horizontal bar chart with month/year X-axis, one row per work item, current date marked with a red line, estimate shown inside each bar - A **craft rollup table** below the timeline aggregates estimates by craft

Empty states: - "No source work synced", source mode, no `replicated_work` rows for this asset - "Asset not yet synced", ArtHound mode, no canonical ID (shouldn't happen post-onboarding) - "No work yet, - "No work yet, use Generate Work", ArtHound mode, synced but no generated work

A `workRefreshKey` prop allows the parent to force a data reload after the scheduler runs.

Reviews Tab (`ReviewsTab.jsx`)

See [Reviews](#) for full documentation. The tab shows a read-only gallery of source-tool attachments at the top, then all ArtHound reviews for the asset below.

Attachments Tab (`AttachmentsTab.jsx`)

Renders the asset's source-tool attachment fields using the copy-on-first-view pipeline. See [Attachment Architecture](#) for full documentation.

Asset Data Model (API response shape)

GET /api/assets and GET /api/assets/{id} return:

```
{
  "id": "recXXXXXX",           // source_record_id
  "canonicalId": "uuid",       // canonical_assets.id
  "name": "Hero Character",
  "devName": "hero_char_v3",
  "itemType": "Character",     // resolved from replicated_item_types
  "product": "Project Atlas",  // resolved from replicated_products
  "productId": "recYYYYYY",    // source_record_id of the product (for drill-down)
  "priority": "P1",
  "projectDate": "2026-06-01",
  "assetNumber": "A-0042",
  "team": "Art",
  "rawFields": {
    "Complexity": "High",
    "Poly Count": 84000,
    ...
  }
}
```

`rawFields` contains all non-hidden meta fields, formatted via `_fmt()` (handles arrays, linked records, select values, booleans). Hidden-tier fields are stripped before the response is sent.

Field Mapping and Meta Bucket Classification

Studios configure field classification in the Field Mapping modal (`FieldMappingModal.jsx`), accessible from the setup/settings area:

Slot assignments: a table mapping Arthound slots (`name` , `dev_name` , `item_type` , etc.) to specific source fields. One source field per slot.

Field classification: for every source field, the studio sets: - **Meta bucket:** Production, Technical, Business, Custom, or Source native - **Display tier:** auto-computed from bucket but overridable - **Skip** (ingest suppressed): field is excluded from `meta` storage entirely

Schema drift: when the source tool's schema changes (fields added, removed, or renamed), `schema_drift_events` records the mismatch. The field mapping modal shows a drift alert listing the changes, and the studio must review and re-save their mappings to dismiss it. The top-bar shows a badge when unresolved drift events exist.

Bucket override log: every manual bucket reassignment is recorded in `field_bucket_override_log` (append-only) for audit purposes.

Reference Cache

`routes/assets.py` maintains an in-process reference cache (`_REF_CACHE`) keyed on `"{owner_type}:{owner_id}"` with a 60-second TTL. The cache stores: - Product name lookup (`source_record_id` → name) - Item type name lookup - Field mapping and tier assignments

This avoids repeated Supabase round-trips on every asset list request. The cache is invalidated on sync completion and on field mapping saves.

Name Write-Back

`PATCH /api/assets/{asset_id}/name` updates the asset's name in the source tool directly (studio-only). Supports both Airtable (PATCH to the record's name field) and Jira (PUT to the issue's summary field). The field ID for the name slot is resolved through `source_field_mappings`, no hardcoding.

Known Gaps

No detail drill-in modal for work items: the Work tab shows a flat list of work items but clicking one does not open a detail view. A work item detail modal is a planned improvement.

No count badge on tab labels: the Work and Reviews tabs do not show a count of items in the tab label. Studios cannot see at a glance how many work items or reviews exist without clicking into each tab.

Asset Viewer UX redesign flagged: the current layout has been flagged as needing a broader UX redesign (recorded in project memory 2026-05-08). Specific pain points are to be gathered before proposing direction.